Rec Room

A 1970s basement games room in Bullet Physics Playground · 20 October 2026

A 1970s basement games room, about 11 by 11 metres, with four playable games in it:

- **Pinball Machine A**, against the back wall, between the bar and the jukebox.
- The **Pool Table**, the **Bumper Pool** table and the **Snooker Table**, one behind another down the right-hand side of the room. Each stands on its own rug with at least 1.5 m all round for the cue, and each has its scoreboard against the right-hand wall.

One of WyomingWill's clocks, **N7**, hangs on the back wall between the bar and the pinball machine, running. The room also has wood panelling, a carpet, a bar with stools and a shelf of bottles, flickering "GAME ROOM" and "POOL" neon signs, a jukebox whose light bars move, a dartboard, two cue racks, a sofa and coffee table across from the tables, a door and wall lamps.

Any other bpp script can be brought in with one line (see "More games"): the room comes with WyomingWill's Jansen walker, shrunk to the size of a cat, walking on its own patch of floor, and koppi's marble run on a small table against the back wall.

The games are the real ones from `../pinball-machine-a/`, `../pool-table/`, `../snooker-table/` and `../bumper-pool/`, loaded unchanged. Everything they do on their own works here too: scoring, sounds, the high scores, saved layouts, the ramp, nudging and tilt, the computer players, and the tables' cameras and aiming guides. All of it keeps running wherever you stand: every computer player and the clock at once.

Running it

Open `rec-room.lua` in bpp. The game folders must be next to this one, as they are in the repository:

```
demo/claude/ pinball-machine-a/ (or Pinball/) pool-table/ (or Pool/) snooker-table/ (or Snooker/)
optional bumper-pool/ (or BumperPool/) optional rec-room/ <- this folder
```

The pinball machine and pool table are required. The snooker and bumper pool tables and the clock are used if they're found; without one, the room goes without it (the console says where it looked). The lists of folder names it tries are near the top of `rec-room.lua`. It works with bpp built on Lua 5.1 or on 5.2 and later.

Keys

Click the 3D view first so it has keyboard focus.

Key	Action
`Tab`	walk to the next spot: the pinball machine, the pool table, the snooker table, the bumper pool table, the clock, each of the "More games" (the marble run, the walker), a view of the whole room, then back to the pinball machine
`Shift+Tab`	walk the other way

On Tab and Shift+Tab the camera flies to the next spot rather than jumping: up over the tables on a smooth curve, in 1.2 to 3.5 seconds, ending on the view the game sets for itself. The game you're going to plays at once; its own camera moves wait until you've arrived. `REC_FLY=0` jumps instead, as before.

Everything else goes to the game you are standing at, and its keys are the same as when it runs by itself (the Shortcuts pane shows them, headed with where you are):

- **At the pinball machine:** `Left Shift` / `Right Shift` (or `Z` / `/`) flippers, `Return` plunger, `Space` shake, `1` new game, `P` computer player, `V` its views, `E` edit the layout. See `../pinball-machine-a/README.md`.
- **At the pool table:** arrows aim and set the force, `W` / `A` / `S` / `D` spin, `Space` shoot, `G` guide, `V` / `B` / `T` cameras, `P` computer player, `N` re-rack. See `../pool-table/README.md`.
- **At the snooker table:** the same keys as the pool table. See `../snooker-table/README.md`.
- **At the bumper pool table:** the same keys again, plus `0` (white: the computer or a second player) and `L` (the computer's level). On its own the table also uses `Tab` to choose your next ball; here `Tab` walks on, so use `X` / `Z`, which do the same. See `../bumper-pool/README.md`.
- **At the clock:** `T` starts tuning its gravity toward 60 s for one turn of the second hand, `G` locks it, `S` turns the tick-tock on or off. Its gravity is the **clock_gravity** slider in the Params pane, and it applies to the clock alone.
- **At one of the "More games":** its own keys, if it has any; its sliders are in the Params pane, named after it (`walker_Speed`). At the walker, the view follows it as it walks.
- **Looking round the room:** the mouse turns and zooms the view as usual.

bpp's own one-letter keys change the whole room: `S` stops the simulation, `D` turns sleeping off (everything is simulated all the time, much slower), `R` reloads the script and `P` saves every frame for POV-Ray. While you're at a game, those four do only what that game uses them for (the clock's `S` for its sound, the tables' `S` and `D` for spin), and nothing at a game that doesn't use them (the walker, the marble run). Looking round the room, they're bpp's as usual, except `P`: saving every frame for POV-Ray writes the whole room to a file each frame and slows it to about two frames a second, and `P` is easy to press by mistake (it's snooker's "computer plays"), so the room view holds it back and says so. Use the POV-Ray menu to save frames from the room.

When you walk away from a game, any keys you are holding are let go (a raised flipper drops; a drawn plunger launches), and the game carries on running. A ball in play on the pinball machine keeps rolling while you are at the pool table, and a computer player keeps playing while you're elsewhere.

The clock

The clock is `N7_Clock_handcheck.lua`, loaded unchanged. The room looks for it in `../Clocks/` or `../../WyomingWill/Clocks/` (where it is in the repository); without it the room has no clock and everything else works.

- **Size:** built full size it is about 4 m tall, so it hangs at a quarter of that. Shrinking is done so the physics is unchanged: every length is scaled by 1/4, its gravity by 1/4 and its motor torques by 1/16, which gives the same motion in the same time. Its gravity slider keeps its usual values: 1166 means what it means when the clock runs on its own.
- **Physics settings:** the clock's escapement only works at the settings it was built with: bpp's usual 100 physics steps a second and its own ERP. At the tables' 900 steps a second it stalls. So the clock gets steps of its own, with the games frozen while it takes them. Everything runs all the time, wherever you're standing: every table's computer player, the pinball's and the clock together. There are two ways of fitting the clock's steps in, set at the top of `rec-room.lua`:
- `OLD_TURNS = false` (the default): every frame, bpp steps the games, then the clock takes whatever steps it's due within the same frame. The games get all 60 frames, at full speed. This needs bpp's `v:stepSimulation` (see "Requirements"); without it the room takes turns anyway. A 67-minute run with shadows on kept the games at 59-60 frames a second, and the clock's 42-beat checks averaged 59.94 s against 60 (all within half a second; no correction needed).
- `OLD_TURNS = true`: the clock and the games take turns, a frame at a time (never two clock frames in a row). The room asks for 83 frames a second, but a 60 Hz screen allows 60, so the games get about 35 of them and run at about 58% speed. This uses the least processor: tuned in the room, the clock averaged 60.10 s and 60.18 s per turn of its second hand on two nights (at G 1210.9 and 1207.3, where training happened to lock), the same clock either way.

- **Pace:** the clock is due a step whenever it has had fewer than 25 for each real second since it started, the pace bpp's frame timer gives it when it runs on its own. The room measures that with a stopwatch (bpp's elapsed-time timer, the one the clock times its own beats with), never the time of day, and it only decides when the clock's world takes a step: nothing ever sets or corrects the hands, wheel or pendulum. So the clock's world runs at the same speed however busy the room is, and a gravity you tune and lock stays right while you play the other games.
- **Hold-ups:** if something holds the room up (a table's computer player can think for up to a second), the clock catches up on the next frames, taking up to 4 steps a frame (`MAX_CATCHUP`). A beat that falls in a hold-up is timed late and the next one early, but none are lost. After a hold-up of more than five seconds (bpp paused, say) the clock lets the rest go and says so in the console.
- **Timing:** the beat is measured against real time. Tuning with `T` at the clock finds the gravity for that.
- **Starting gravity:** the room starts the clock at G 1215 (`CLOCK_G` at the top of `rec-room.lua`), where it keeps 60.0 s a turn: three overnight runs averaged out at 1215, 1215 and 1214.7. The clock's own script starts at 1166. The slider and tuning work as usual from there; `CLOCK_G = nil` starts it where its script says.

More games

Any bpp script can join the room with one line in `MORE_GAMES`, near the top of `rec-room.lua`. The room comes with two:

```
lua MORE_GAMES = MORE_GAMES or { { "../../koppi/marblerun.lua", on = "table", x = 250, z = -95, size = { 70, 50 }, scale = 2, ground = -0.5, awake = true, solid = false, name = "the marble run" }, { "../Walkers/Jansen_6LegT.lua", on = "floor", x = -40, z = 330, size = { 300, 110 }, scale = 0.12, height = 2, solid = false, follow = true, name = "the walker" }, }
```

The first thing on a line is the script, from this folder (or a full path). The rest is optional:

Setting	What it does
`on`	what it stands on: `"floor"` (a patch of floor, a mat), `"table"` (a table the room puts there) or `"wall"` (a shelf on the wall nearest `x, z`, its back to the wall, facing into the room). Default `"floor"`.
`x, z`	where its middle goes, in cm. The room runs from x = -470 (left wall) to 650 (right wall), and from z = -150 (the back wall, behind the pinball machine) to 960 (the door's wall).
`size`	`{ w, d }`: the floor patch, table top or shelf, in cm. Default `{ 100, 60 }`.
`height`	the table top or shelf, in cm above the floor (default 75 for a table, 110 for a shelf); for a floor game, how far its own floor sits above the mat
`turn`	degrees, anticlockwise seen from above. 0: its front (+z) faces the door. On a wall it faces into the room unless you say otherwise.
`scale`	its size (0.12: an eighth or so). Its gravity and motors are scaled with it, so it moves as it does full size, as the clock does.
`ground`	the height of its own floor in its own units (default 0): that is what rests on the floor, table or shelf
`name`	what Tab calls it; its sliders are named after it
`awake`	`true`: its parts never go to sleep, for a demo that asks you to turn deactivation off (the marble run)
`solid`	`false`: the table top, shelf or mat is only to look at. For a game with a floor of its own (the walker's terrain) or none it needs (the marble run). A solid top is tested against every moving part near it, on every step: under the marble run's wheel it tripled its cost.
`follow`	`true`: while you're at it, the view follows it as it moves
`keep`	`true`: never put back (see below)

How the room fits a game in:

- **Its own physics.** Each runs with its own physics settings, at its own pace (the frames it gets on its own, against a stopwatch), in steps of its own between the games' frames, exactly as the clock does. So the marble run turns at

the speed it does on its own, and the walker walks as it does on its own.

- **Its ground.** A script's ground plane (`Plane`) would be endless in the room, so it's left out; the room's floor patch, table or shelf stands in for it. A script's terrain (`Terrain`, the walker's) is moved, turned and scaled with it and cut to its patch of floor (the walker's is 100 m square on its own).
- **Putting back.** When a part of it leaves its floor patch, table or shelf (falls off, walks off), the room puts the game back as it started: every part it built as it loaded goes back where it was then, at rest, and its script carries on. Something it made later, such as one of the marble run's marbles, is just taken away if it falls. Nothing is deleted and made again, so a room left running all night uses no more memory for it (300 put-backs in a test: no growth at all). The walker gets put back about every 80 seconds, after walking about 1.2 m. If a game is put back twice within 5 seconds, it doesn't fit (its `size` is too small), and the room leaves it as it is and says so in the console.
- **Its fixed parts asleep.** Bullet puts fixed parts (mass 0) to sleep, so two of them are never tested against each other. The walker's script keeps its terrain awake, which made it be tested against the mat on every step of every game: the clock's steps took five times as long. The room puts a game's fixed parts back to sleep after it loads; they still stop whatever hits them.
- **Its cost.** Each shows on the meter's physics line. In tests the marble run took about as long as on its own plus a little for the room, the walker about 1 ms a frame. Both together add a few milliseconds a frame, so with shadows on, a room that was at 83% may now run near 100%; the games then slow down a little, and the clock keeps time. Remove a line (or comment it out with `--`) to leave a game out.

With `OLD_TURNS = true` they run with the tables' physics settings instead of their own, and may not behave as they do on their own.

The cost meter

Every 10 seconds (`METER_PRINT`) the console shows what each frame spends its time on, averaged over the last second, in milliseconds per frame:

```
COST METER -- ms per frame, averaged over the last second 60 frames a second: 35 for the games, 25 clock steps; longest frame 19 ms physics games 1.55, clock 1.70 scripts pinball 0.10, pool 0.35, snooker 0.30, bumper pool 0.20, clock 0.02, room 0.45 garbage 1.20 drawing 3.30 thinking up to 3.0 drawn 575 objects, 1188 into the shadow map busy 8.8 ms of each 16.7 ms frame (53%)
```

- **frames a second:** the room's, how many of them stepped the games, and how many steps the clock took (which should be 25).
- **longest frame:** the longest wait between two frames shown in that second. Anything that holds the room up (a computer player thinking, a big garbage collection, a heavy break) shows here; a smooth room keeps it near 17 ms on a 60 Hz screen.
- **physics:** Bullet's time stepping the games, the clock, and each of the "More games". (The games share one physics world, so they can't be split game by game.)
- **scripts:** each game's own Lua code, including its computer player, and the room's (taking turns and so on).
- **garbage:** Lua's garbage collector, which collects a little every frame (see "Garbage" below).
- **drawing:** bpp drawing the scene. Time the graphics card spends after that, and waiting for the next frame, isn't counted.
- **drawn:** on a bpp that skips what can't be seen (`v.culling`, on by default), how many of the room's 3,300 or so objects the last frame drew for the screen, and, with shadows on, into the shadow map: only what's in view, or could throw a shadow into it. A bpp that records the fixed objects' shadows (`v.shadowCache`, on by default) adds "and N more from its record": those it drew into the shadow map once and replays each frame, instead of drawing them one by one. A bpp that also saves their depth (`v.shadowSaved`, on by default) says "from its saved depth" instead: it drew them into the shadow map once, and copies that back each frame, which costs the graphics card almost nothing.

- **timing:** only with bpp's drawing timer on (`v.drawTiming = true` before loading the room). It splits the drawing time into working out the scene's extent ("box"), deciding what to skip ("cull"), the shadow map, and the screen pass, as the processor spends it, and the shadow map and the screen pass as the graphics card spends it (from its own timers). "All of draw" is the whole of bpp's drawing, which should match "drawing" above.
- **thinking up to:** how long the tables' computer players may think each frame (see "The computer players' thinking" below).
- **busy:** all of the above, against the time each frame actually has (16.7 ms on a 60 Hz screen, which holds the room to 60 frames a second whatever it asks for). Near 100% the room can't keep up, and the meter says so; it then suspends a game (see "Suspending games" below), and a "suspended" line names those it has. The clock keeps its pace as long as it can fit its steps in.

While the simulation is paused, the meter says so and shows only the drawing and garbage time per drawn frame (bpp keeps drawing).

The same lines go to the console every 10 seconds, so they can be copied. And any frame that takes longer than a quarter of a second (`FREEZE_LOG_MS`, 250 ms; 0 turns it off) is noted in the console with the time of day it ended and where that frame's time went, for example

```
REC ROOM: a 454 ms freeze, ending at 18:38:41 -- physics 3, garbage 4, drawing 32, outside the room's code 413 ms
```

With bpp's drawing timer on (`v.drawTiming = true`), the line ends with that frame's drawing part by part, and anything it had to do besides drawing:

```
[drawing: box 1.2, merge 0.3 (2 recoloured), cull 0.9, shadow map 14.1 (record of 46 fixed objects made again, saved depth made again), screen 3.4, all 19.9]
```

A game named there (`snooker 380`, say) held the room up itself; time "outside the room's code" went to bpp's window, the graphics driver or the system, and the time of day lets it be matched with the system's own log. (The time of day is only printed, never used for timing.) The first 10 seconds, while the room loads, are left out. Set `METER_PRINT = 0` at the top of `rec-room.lua` to stop that, or `METER = false` to turn the meter off.

The meter, and the room's garbage allowance, time short pieces of work with bpp's stopwatch (`v:getTime`) when it is fine-grained: a bpp whose `getTime` returns a double precision number from `nsecsElapsed`. An older bpp's stopwatch ticks in whole milliseconds and, being single precision, ever more coarsely the longer bpp runs (every 2 ms after 4.6 hours, every 8 ms after 18). On such a bpp the room says so in the console and times the work with the processor clock instead. Otherwise the garbage allowance would overrun more and more through a long run (an overnight run saw the room's processor use climb from 65% to 90%), and the meter's late figures would be meaningless.

Suspending games

Everything runs all the time, wherever you stand, as long as the room can do it all in a 60th of a second. When it can't (shadows on and every game playing itself, say, or more games than the computer can keep up with), it suspends a game: the most expensive one just then, but never the clock and never the game you're at. A suspended game stops exactly where it is, with a PAUSED sign over it, and carries on from there when there's room again, or the moment you walk over to it. Each change is noted in the console:

```
REC ROOM: the walker suspended (about 2.3 ms of each frame just then) -- the room needed 17.8 ms a frame, ... REC ROOM: the walker resumed (the room has room for it again: 12.2 ms a frame without it, and it usually takes about 1.1)
```

So more games can be added to the room without slowing it down: at a game, only that game and the clock need to run; looking round the room, the busiest of the rest wait their turn.

- **When.** Once a second the room measures its work per frame (the tables' computer players' thinking counted at its usual cost: they think with whatever time is spare; and drawing at its usual cost, the least of the last 10 seconds: a spike in drawing costs the same whether a game runs or not, so suspending one wouldn't help). Over `SUSPEND_ABOVE` (15.5 ms) for `SUSPEND_AFTER` (2) seconds running, the most expensive game just then is suspended. With the cheapest suspended game added back at its usual cost, under `RESUME_BELOW` (15 ms) for `RESUME_AFTER` (5) seconds running, it's resumed. After each change the room waits two seconds before the next. `AUTO_SUSPEND = false` turns it off.
- **A game's cost.** Its scripts and its physics. The clock and the "More games" are timed stepping on their own; the pinball machine and the tables share one step, so that is shared out by how many of their bodies are moving. Whether a game fits back is judged by its usual cost, an average over about the last 20 seconds it ran, not by what it cost at the moment it was suspended (a table in the middle of a break costs several times what it does waiting for its next shot).
- **While suspended.** Its moving parts are taken out of the simulation and its scripts aren't called, its computer player included. A game with its own physics isn't owed the time it was away. A few of the games' timers run on the real clock (the pause before a new rack, say), so one that came due while it was away happens as soon as it's resumed.

How it works

`rec-room.lua` runs each game in its own sandbox (a Lua environment of its own) so the scripts can't interfere with each other:

- **Position.** Each game is moved to its place in the room (the clock is also shrunk). Positions it reads or sets are converted between its own coordinates and the room's, so the game's code doesn't need to know it has been moved.
- **Gravity.** The tables use ordinary gravity. The pinball playfield uses gravity tilted by 6.5°, and the clock whatever its slider says, each set on its own bodies only.
- **Camera, keys and news.** A game only controls the camera and the Shortcuts pane (its keys) while you are standing at it, and only its news (whose turn, the score) goes to the console; walking over to a game shows where it stands.
- **Callbacks.** Each game's per-frame and key callbacks are collected, and the room calls them in turn.
- **Files.** The file each game opens, and each sound or mesh it loads, is looked up in that game's own folder.
- **Physics.** The pinball machine and the tables share a 1/900 s physics step, which the tables need for their breaks. The pinball flippers scale their motor strength to the step, so they flip exactly as they do on their own. The clock takes turns with them at its own settings (see above). Each game counts only its own frames, so its timers work as they do on their own.
- **One step of its own a frame.** The clock and the "More games" each take their own physics steps, 25 a second, a few milliseconds each. In a frame where two fell together the frame missed the screen, so the room takes one of them a frame where it can: the clock first, then the one furthest behind; the next waits a frame (16 ms) unless it's two steps behind. Each still takes exactly its steps, at its own pace.
- **Keys and sliders.** A game's keys, its keyboard shortcuts included, go to it only while you're at it. Its sliders are renamed in the Params pane when needed (the clock's gravity is `clock_gravity`).
- **Resting tables.** The tables keep their balls awake on purpose, so they'd be solved 900 times a second even sitting still. Each table takes its balls out of the simulation itself while it waits for a shot with every ball still, and puts them back as the stroke begins.
- **Garbage, on a bpp that collects it itself.** A bpp built with the ownership fix (Oct 2026) keeps Lua's garbage collector running and sets `BPP_GC_AUTO`. The room then leaves garbage alone: the collector works a little at a time as the scripts make garbage (its time shows in theirs, and the meter's "garbage" figure stays near 0), and the games skip their own collections. Everything below applies only to an older bpp.

- **Garbage.** bpp keeps Lua's garbage collector stopped, and each game normally collects everything every couple of seconds. The games share one Lua heap here, so each of those collections would clear the whole room's garbage at once, a noticeable pause. Instead the room collects a little every frame, for up to 2 ms (`GC_BUDGET`), and the games' own full collections are skipped. Setting `GC_STEADY = false` makes it wait until the heap has grown by half before collecting. That costs less on average, but the end of a collection can't be split up, and with that much to finish it held the room up for 50 to 110 ms every few seconds (on an overnight run, before the tables' computer players made less garbage, a few tenths of a second about every 9 seconds).
- **The computer players' thinking.** The pool, snooker and bumper pool tables' computer players think a few milliseconds a frame and pick up where they left off on the next (see each table's README). Before, a snooker shot could hold the whole room up for over a second. With every computer player on, a test run of 10,000 frames went from 51 frames over a tenth of a second (the longest 2.3 s) to none (the longest 41 ms). On their own they think 3 ms a frame. Here they get what the frame has to spare: each second the room works out what everything else took and lets them think for the rest of 90% of a 60th of a second (`THINK_FILL`), between 0.5 and 3 ms (`THINK_MIN`, `THINK_MAX`). With shadows on, the room was at about 83%, and the 3 ms tipped it over now and then (14 seconds in 67 minutes, every one while bumper pool's computer was thinking). Now a busy room makes the computer think longer (bumper pool's longest shots, about a quarter of a second of thinking, then take up to 6 seconds) instead of slowing the room down.

The room's own furniture is scenery only; it doesn't collide with anything.

The room has its own light, high up over the left-hand side (`ROOM_LIGHT`, x, y, z over w as OpenGL takes it). bpp's usual light is off to the right, outside the right-hand wall, so with shadows on the wall and the scoreboards shaded the far half of each table. The room's comes down steeply enough that the left-hand wall's shadow stops short of the pinball machine and the clock. Set `ROOM_LIGHT` before loading the room to put it somewhere else (`btVector4(500, 500, 500, 0.4)` is bpp's usual one). It also lights the scene POV-Ray renders.

Requirements

- bpp with the keyboard hook (`v:onKey`) and `v:playSound`, which the games need as well.
- For a smooth frame rate, bpp with cached drawing (this commit's changes to `src/objects/mesh.cpp` and `src/glutils.cpp`): each mesh, and each sphere, cylinder, cone and cube shape, is compiled into an OpenGL display list once and replayed every frame, instead of being sent a triangle at a time. The clock's gears alone are well over 100,000 triangles. The room runs without it, only using more of the processor.
- bpp with `v:stepSimulation` (a change to `src/viewer.cpp` and `src/viewer.h`), so the room can step the clock and the "More games" within the games' frames (see "The clock"). Without it the room takes turns (as with `OLD_TURNS = true`).
- Also for speed, bpp that brings only moved objects' bounding boxes up to date (this commit's change to `src/viewer.cpp` and `src/viewer.h`). Bullet's default recomputes every object's box on every physics step, moving or not. With 900 steps a second and thousands of fixed parts (cushions, rails, scoreboard digits, scenery), that was most of the physics time at rest. Now Bullet updates only moving objects, and before each frame bpp updates any fixed or sleeping object a script has moved.
- OpenSCAD, for the clock's gears (as the clock needs on its own).
- The game folders beside this one, at least at the versions in this commit. The pinball's flipper strength now follows the physics step, and older copies would flip weakly in the room.